

WeTravellers

Travel Provider + Market + Payment Architecture

Agent Implementation Specification - Version 2.0 (Final Agent Handoff) | 24 August 2026

Scope: Multi-provider flights/hotels/cars, market-aware pricing, country/currency selection, payment routing, settlement, booking reliability, security, tests, and production readiness.

Decision	Current Choice
Flights	Duffel as first implementation; WINGIEONE kept as competitive/backup provider candidate.
Hotels	Nuitee Connect as first implementation.
Cars	Provider intentionally not chosen yet; must implement via the same CarProvider interface.
Customer payment	WeTravellers owns the payment abstraction. Egypt starts with a local Egyptian gateway; other markets are plug-ins.
Provider settlement	Separate from customer payment. Duffel/Nuitee use their own settlement methods; never mix display currency with provider settlement currency.
Architecture	Backend owns providers, pricing, booking, payments, webhooks, secrets, ledger, reconciliation. Flutter consumes unified domain APIs.
Non-negotiable	No provider-specific code in Flutter; no production secret in the app; no direct client -> supplier API calls.

Important execution rule: this document is a build specification, not permission to delete or replace existing project files. Preserve the current repository, architecture, tests, and unrelated modules. Changes must be incremental and reviewable.

Validation rule: every phase must end with tests. For Flutter: flutter analyze + flutter test. For backend: unit/integration/contract tests appropriate to the changed modules. Never mark a phase complete because code compiles only.

Document Map

- A. Executive decisions and non-negotiable rules
- B. Target architecture
- C. 52-point implementation specification
- D. Execution phases and installation order
- E. Provider onboarding/runbooks
- F. Testing strategy and production gates

G. Open questions that require provider confirmation

H. Research notes and official sources

A. Executive decisions and non-negotiable rules

- Provider-agnostic domain: FlightOffer, HotelOffer, CarOffer, Booking, Payment, TripItem, MarketContext are internal contracts.
- Provider adapters translate third-party schemas into the internal domain; provider IDs are stored as source metadata, never used as UI/domain truth.
- Market selection is not a provider selector. MarketContext controls POS/market, display currency, locale, payment region, tax region and other market rules.
- Customer currency and provider settlement currency are separate values. Never force a provider to accept the UI currency unless that provider explicitly supports it.
- Search prices are provisional. A selected offer must be revalidated/prebooked before payment and final booking.
- All money-changing operations are idempotent. Payment success + booking timeout is a first-class recovery state.
- Webhooks are persisted before processing, deduplicated, verified, and mapped to internal events.
- Provider credentials, payment secrets and webhook secrets live in a secret manager or protected environment, never in Flutter.
- Finance is a first-class domain: payment attempts, provider charges, fees, FX, markup, refunds, wallet top-ups and reconciliation must be recorded.
- Production activation must be feature-flagged and market-scoped, so a provider can be enabled for one market before global rollout.

B. Target architecture

Flutter

|

v

Backend API / BFF

|

+--> Auth / User / Trip

|

+--> Market Service

| |-- MarketContext

| |-- POS rules

| |-- Currency / locale

| |-- Payment region

|

+--> Search Orchestrator

| |-- FlightProvider(s)

| |-- HotelProvider(s)

| |-- CarProvider(s)

| |-- normalize / dedupe / rank

|

+--> Pricing / FX Service

|

+--> Booking Service

| |-- revalidate / prebook

| |-- payment state

| |-- provider booking

| |-- cancel / change / refund

|

+--> Payment Orchestrator

| |-- Egypt gateway

| |-- Saudi gateway

| |-- UAE gateway

| |-- future gateways

|

+--> Settlement / Ledger / Reconciliation

|

+--> Webhook Gateway

|

+--> Notifications

|

+--> Admin / Provider Config / Feature Flags

External:

Duffel, Nuitee, future car provider(s), payment gateways, FX source(s), messaging providers

Core implementation boundaries

Layer	Owns	Must not own
Flutter	UX, presentation, local state, user input, trip display	Supplier keys, pricing authority, booking authority, payment secrets
API/BFF	Auth, request validation, response shaping	Provider business rules
Domain	Unified travel/payment/booking models and state machines	Third-party JSON schemas
Provider adapter	Mapping, auth, provider-specific retries, capabilities, raw IDs	UI concerns, customer wallet rules
Pricing	FX, markup, fees, final display amount	Changing provider's source amount silently
Payment	Payment intent/attempt/refund orchestration and gateway routing	Provider inventory logic
Finance	Ledger and reconciliation	UI formatting
Webhook	Verification, dedupe, ingestion, event mapping	Directly mutating unrelated domains without event handling

C. 52-point implementation specification

1. Provider layer

الهدف

Create stable FlightProvider, HotelProvider and CarProvider interfaces. The app must call these interfaces, never vendor SDKs or endpoints.

طريقة التنفيذ

Create packages/modules such as backend/src/modules/travel/providers/{flights,hotels,cars}. Each provider adapter implements the same internal methods and returns domain objects. Include provider metadata on every result: providerKey, providerResourceId, retrievedAt, expiresAt.

ملاحظات/قرارات

Do first, before any live vendor integration. This is the main anti-lock-in boundary.

معياري القبول

Acceptance: replacing a provider does not require Flutter changes or domain-model rewrites.

```
interface FlightProvider {  
  capabilities(): ProviderCapabilities  
  search(request: FlightSearchRequest): Promise<FlightSearchResult>  
  revalidate(input: RevalidationRequest): Promise<RevalidatedOffer>  
  book(input: FlightBookRequest): Promise<BookingResult>  
  cancel(input: CancelRequest): Promise<CancelResult>  
}
```

2. Capability model

الهدف

Represent feature support per provider because providers differ.

طريقة التنفيذ

Create ProviderCapabilities with search, prebook, hold, book, cancel, refund, change, seats, baggage, webhooks, payment, images, reviews and other product-specific capabilities.

ملاحظات/قرارات

Routing must verify capability before selecting a provider. Unsupported operations return a domain capability error, not a generic 500.

معياري القبول

Acceptance: tests prove a provider lacking cancel cannot be used for a cancellation workflow.

3. MarketContext

الهدف

Centralize country, display currency, locale, payment region, POS and tax region.

طريقة التنفيذ

Create MarketContext value object and Market configuration tables. Initial market: EG, EGP, ar-EG, paymentRegion EG.

ملاحظات/قرارات

Do not use a single `country` field for every purpose.

معييار القبول

Acceptance: switching market updates localization and payment routing without changing provider code.

```
ProviderCapabilities {  
  search, revalidate, prebook, hold, book,  
  cancel, refund, change, seats, baggage,  
  webhooks, payment, images, reviews  
}
```

4. Country semantics

الهدف

Separate market/POS country, payment country, residence country, nationality, origin country, destination country and hotel country.

طريقة التنفيذ

Use explicit fields in relevant requests. Keep user profile fields separate from trip/search fields.

ملاحظات/قرارات

Do not infer paymentCountry from nationality or residence.

معييار القبول

Acceptance: an Egyptian resident paying from UAE can be represented without losing the Egypt nationality/residence distinction.

```
MarketContext {  
  marketCountry: ISO-3166-1 alpha-2  
  displayCurrency: ISO-4217  
  locale: BCP-47  
  paymentRegion: ISO-3166-1 alpha-2  
  posCountry: ISO-3166-1 alpha-2 | null  
  taxRegion: string | null  
}
```

5. Currency model

الهدف

Separate provider, display, payment and settlement currencies.

طريقة التنفيذ

Store providerAmount/providerCurrency, customerAmount/customerCurrency, paymentAmount/paymentCurrency, settlementAmount/settlementCurrency where applicable.

ملاحظات/قرارات

Never mutate the provider amount to match display currency; conversion is a presentation/pricing layer.

معييار القبول

Acceptance: a Duffel offer in GBP can be shown in EGP while the provider charge remains GBP.

6. FX service

الهدف

Provide auditable FX conversion with source, timestamp and rate ID.

طريقة التنفيذ

Create FxService and FxRate entity. Cache rates, include timestamp and source. For booking, store the FX snapshot used to quote the customer.

ملاحظات/قرارات

Do not use a live FX rate during refund/reconciliation without retaining the original snapshot.

معياري القبول

Acceptance: a historical booking can reconstruct the quoted customer price from stored FX data.

```
Money {  
  amount: decimal  
  currency: ISO-4217  
}  
PriceSnapshot {  
  provider: Money  
  customer: Money  
  payment: Money  
  settlement: Money | null  
  fxRateId: string | null  
  capturedAt: instant  
}
```

7. Price provenance/expiry

الهدف

Know where every price came from and when it stops being trusted.

طريقة التنفيذ

Every offer stores provider, providerOfferId, retrievedAt, expiresAt, sourceCurrency and sourceAmount.

ملاحظات/قرارات

Search result must not be treated as bookable just because the UI still displays it.

معياري القبول

Acceptance: expired offers force a refresh/revalidation rather than payment.

8. Revalidation / prebook

الهدف

Final price and availability must be checked immediately before payment/booking.

طريقة التنفيذ

Implement a standard workflow: SELECTED -> PRICE_REVALIDATED. Map Duffel's re-check/order rules and Nuitee's Rates -> Prebook -> Book into this internal state.

ملاحظات/قرارات

If price changes, return a structured PRICE_CHANGED result with old/new breakdown; never silently charge the new amount.

معييار القبول

Acceptance: provider price-change fixtures block payment until the user accepts the new amount.

```
PriceQuote {
  lines: [
    BASE,
    TAX,
    PROVIDER_FEE,
    PAYMENT_FEE,
    MARKUP,
    DISCOUNT
  ]
  customerTotal
  providerTotal
  currency
  expiresAt
  pricingVersion
}
```

9. Booking state machine

الهدف

Unify provider-specific states into a safe internal state machine.

طريقة التنفيذ

Implement states such as DRAFT, SEARCHED, SELECTED, PRICE_REVALIDATED, PAYMENT_PENDING, PAYMENT_SUCCEEDED, BOOKING_PENDING, CONFIRMED, BOOKING_FAILED, CANCEL_PENDING, CANCELLED, REFUND_PENDING, REFUNDED, RECONCILIATION_REQUIRED.

ملاحظات/قرارات

Provider states must map into this state machine; do not expose vendor enums directly to Flutter.

معييار القبول

Acceptance: every terminal state has a recovery path or explicit human-support path.

10. Idempotency

الهدف

Prevent duplicate charge, duplicate booking, duplicate cancel and duplicate refund.

طريقة التنفيذ

Store idempotencyKey per business operation and provider attempt. Return the original result for duplicate requests.

ملاحظات/قرارات

Idempotency key must be bound to operation type and authenticated user/booking scope.

معييار القبول

Acceptance: sending the same Book request 3 times creates at most one external booking.

```
Search -> Select -> Revalidate/Prebook -> Freeze customer quote
      -> Payment -> Book -> Confirm
```

11. Webhook gateway

الهدف

Ingest provider/payment notifications reliably.

طريقة التنفيذ

Create a single webhook ingress that validates signatures, stores raw payload, deduplicates, and publishes internal events.

ملاحظات/قرارات

Never rely only on webhooks; have provider query/reconciliation fallback.

معييار القبول

Acceptance: duplicate and out-of-order webhook fixtures do not corrupt booking state.

```
BookingState:
DRAFT | SELECTED | PRICE_REVALIDATED | PAYMENT_PENDING |
PAYMENT_SUCCEEDED | BOOKING_PENDING | CONFIRMED |
BOOKING_FAILED | CANCEL_PENDING | CANCELLED |
REFUND_PENDING | REFUNDED | RECONCILIATION_REQUIRED
```

12. Search orchestration

الهدف

Run one or multiple providers and return a unified search result.

طريقة التنفيذ

SearchOrchestrator calls eligible providers in parallel, normalizes results, attaches source metadata and errors, then ranks/deduplicates.

ملاحظات/قرارات

Partial provider failure must not fail the whole search unless no eligible provider succeeds.

معييار القبول

Acceptance: provider A timeout + provider B success returns B results with observability data.

```
IdempotencyRecord {
  key
  operation
  userId
  bookingId
  requestHash
  firstResult
  status
  createdAt
}
```

13. Deduplication

الهدف

Prevent duplicate travel products across providers.

طريقة التنفيذ

Create canonical keys. Flights: carrier/flight number/times/origin/destination plus segment signature. Hotels: mapped hotel ID where available, otherwise normalized name/geo heuristics. Cars: supplier/location/category/vehicle signature.

ملاحظات/قرارات

Keep each provider offer as a distinct child offer even when grouped into one canonical product.

معييار القبول

Acceptance: duplicate provider results collapse visually while preserving source-specific booking IDs.

```
POST /webhooks/:provider
1. verify signature
2. persist raw event
3. dedupe by provider event id
4. publish internal event
5. acknowledge fast
6. process asynchronously
```

14. Ranking

الهدف

Provide Cheapest, Best, Fastest and Recommended modes.

طريقة التنفيذ

Create RankingService with transparent features: total cost, duration, stops, baggage, refundability, provider health, and optional business margin rules.

ملاحظات/قرارات

Do not secretly bias `Cheapest` by markup. Keep commercial recommendation separate from factual lowest-price ranking.

معييار القبول

Acceptance: Cheapest is truly lowest customer total among bookable offers.

```
Promise.allSettled([
  Duffel.search(...),
  OtherFlightProvider.search(...)
])
-> normalize
-> dedupe
-> rank
-> return partial-success response
```

15. Provider health

الهدف

Detect degraded providers and stop routing new traffic when appropriate.

طريقة التنفيذ

Create health/latency/error metrics and ProviderHealth state. Use circuit breaker/backoff for repeated failures.

ملاحظات/قرارات

Health is capability- and market-aware; a provider may be healthy for EG flights but unavailable for another market/product.

معياري القبول

Acceptance: repeated 5xx responses trip the circuit without permanently disabling the provider.

16. Provider routing/failover

الهدف

Allow primary/secondary providers per product and market.

طريقة التنفيذ

Routing policy includes priority, weight, capabilities, market eligibility, health and commercial enablement.

ملاحظات/قرارات

Never fail over after money is captured unless the booking workflow explicitly supports recovery; provider switching is safe mainly before payment or through reconciliation.

معياري القبول

Acceptance: search failover works; booking failover requires explicit idempotent/recovery rules.

17. Provider registry/config + no hard-coded routing

الهدف

Centralize provider metadata and activation. Provider choice must be configuration/routing, not if/else logic spread throughout services.

طريقة التنفيذ

Store provider key, product, environment, enabled, priority, markets, capabilities, credential reference, webhook reference and feature flags. Create ProviderRegistry and ProviderRoutingPolicy. Routing takes productType, market, capability, environment, provider health, and priority.

ملاحظات/قرارات

Secrets stay outside ordinary DB/config UI. Do not route by user country alone. Country may affect eligibility, but provider capability and commercial configuration are separate.

معياري القبول

Acceptance: Admin can disable Duffel search for EG without a redeploy. Acceptance: enable/disable/reorder providers from configuration without code branching in use cases.

18. Provider adapter/mapping layer

الهدف

Translate third-party JSON/XML to stable internal models.

طريقة التنفيذ

Each adapter has request mapper, response mapper, error mapper, capability descriptor and contract tests.

ملاحظات/قرارات

Do not let raw vendor payloads leak into domain code except stored raw-event/raw-response audit data.

معيـار القبول

Acceptance: internal domain tests can run against fixtures without network access.

```
RoutingRule {  
    product  
    market  
    provider  
    enabled  
    priority  
    weight  
    capabilities[]  
    environment  
    fallbackRank  
}
```

19. Environment isolation

الهدف

Separate dev/test/staging/production per provider and payment gateway.

طريقة التنفيذ

Use explicit environment selectors and secret namespaces: DUFFEL_TEST/LIVE, NUITEE_TEST/LIVE, GATEWAY_TEST/LIVE, etc.

ملاحظات/قرارات

Never infer environment from API key prefix alone in business logic; configuration must be explicit.

معيـار القبول

Acceptance: production build cannot load sandbox secrets.

```
ProviderConfig {  
    key  
    product  
    implementation  
    enabled  
    environments  
    markets[]  
    priority  
    capabilities[]  
    credentialRef  
    webhookSecretRef  
}
```

20. Secrets management

الهدف

Protect supplier keys, gateway secrets, webhook signing keys and wallet credentials.

طريقة التنفيذ

Use environment secret manager or protected CI/CD secret store. Backend only.

ملاحظات/قرارات

Flutter must never ship supplier credentials.

معييار القبول

Acceptance: secret scans fail CI when a provider token is committed.

```
Adapter =  
  requestMapper  
  responseMapper  
  errorMapper  
  capabilityDescriptor  
  rawPayloadSanitizer  
  contractFixtures
```

21. Flight provider - Duffel

الهدف

Implement Duffel as first FlightProvider.

طريقة التنفيذ

Start in test mode. Implement Offer Requests, offer selection, services/seats when supported, order creation, payment strategy, cancellation/change flows, webhook handling, and error mapping. Keep provider currency and customer currency separate.

ملاحظات/قرارات

Duffel test mode is a real sandbox. Duffel Airways offers predictable behavior but unrealistic schedules/prices; dedicated route fixtures cover hold orders, connecting flights, payment 200/202 scenarios, and test card behaviors.

معييار القبول

Acceptance: full sandbox flight smoke test passes from search to order and cancellation.

```
Environment:  
DEV -> provider sandbox  
TEST -> provider sandbox + deterministic fixtures  
STAGING -> provider sandbox/live-like  
PROD -> provider live only
```

22. Duffel test routes and payment

الهدف

Use predictable Duffel test scenarios to prove failure handling.

طريقة التنفيذ

Build fixtures for no flights PVD->RAI, hold JFK->EWR, connecting LHR->DXB, card payment scenarios LTN->STN and SEN->STN, plus test cards and 3DS.

ملاحظات/قرارات

Treat 200/202 payment responses as asynchronous recovery cases and query/order-status + webhook rather than assuming immediate failure/success.

معيار القبول

Acceptance: tests explicitly cover delayed order visibility and failed payment.

Secrets:

PROVIDER_API_KEY

PROVIDER_WEBHOOK_SECRET

PAYMENT_GATEWAY_SECRET

WALLET/CARD_REFERENCE

All resolved server-side from secret storage.

23. Duffel production/go-live

الهدف

Move to live only after sandbox, account activation and balance setup.

طريقة التنفيذ

Use a live token after Duffel account activation/verification. If using Balance settlement, fund the Balance via the supported top-up process. Platform fees are billed to the card on file per Duffel's onboarding flow.

ملاحظات/قرارات

Do not activate live bookings until reconciliation and refund flows are tested.

معيار القبول

Acceptance: live feature flag can be enabled only for an allowlisted market/provider.

DuffelAdapter:

createOfferRequest

getOffer

createOrder / hold

addServices/seats where supported

createPayment (strategy-specific)

cancel/change

webhook mapping

24. Duffel pricing/fees

الهدف

Model provider fees separately from supplier fare and WeTravellers markup.

طريقة التنفيذ

Current published Duffel Flights pricing includes \$3 per confirmed order, 1% managed content on confirmed order value, and \$2 per paid ancillary. Search overage rules must be kept configurable because vendor pricing can change.

ملاحظات/قرارات

Do not bake published pricing into customer pricing code; store provider_fee rules/config so they can change without a release.

معياري القبول

Acceptance: booking ledger stores supplier amount, Duffel fee, payment fee, FX and customer markup independently.

25. Hotel provider - Nuitee

الهدف

Implement Nuitee as first HotelProvider.

طريقة التنفيذ

Implement hotel content, images, room details, rates/search, prebook, book, cancellation where supported, webhooks, and provider error mapping.

ملاحظات/قرارات

Nuitee's core booking workflow is Rates -> Prebook -> Book. Sandbox key requires no credit card and sandbox uses an attached hidden test account card for ACC_CREDIT_CARD.

معياري القبول

Acceptance: sandbox hotel search -> room -> prebook -> book succeeds and appears in provider dashboard.

26. Nuitee hotel content/images

الهدف

Use provider content for hotel cards/details, not a second pricing provider.

طريقة التنفيذ

Cache static content such as hotel name, address, facilities, ratings and images. Use mapped room IDs when room photos/details are required.

ملاحظات/قرارات

Never assume every hotel has a full gallery. Treat photos as optional arrays and preserve source URLs/attribution rules.

معياري القبول

Acceptance: hotel detail renders correctly when images are empty, one image, or many images.

27. Nuitee payment strategy

الهدف

Start with WeTravellers as Merchant of Record and local customer payment, while supporting a switch to Nuitee User Payment later.

طريقة التنفيذ

Define HotelSettlementStrategy: OUR_MERCHANT_OF_RECORD uses our gateway then Nuitee ACC_CREDIT_CARD/Wallet; NUITEE_MOR uses Nuitee Payment SDK. Keep both behind a feature flag/config.

ملاحظات/قرارات

Nuitee docs explicitly support ACC_CREDIT_CARD for apps that implement their own payment layer, including bundling/markup scenarios. Production requires production key, account credit card and wallet funds.

معيار القبول

Acceptance: booking service can choose strategy without changing hotel domain models.

```
NuiteeAdapter:  
  hotel content  
  hotel rates  
  prebook  
  book  
  cancel  
  webhook mapping  
  ACC_CREDIT_CARD settlement strategy
```

28. Nuitee commission/markup

الهدف

Control hotel commercial margin without corrupting supplier pricing.

طريقة التنفيذ

Use internal PricingEngine for our-MOR path; if using Nuitee commissionable rates, map margin explicitly and store the provider's terms. Store checkout price and supplier amount separately.

ملاحظات/قرارات

Nuitee documents margin/commission and weekly payouts for confirmed stays; payout mechanics for a specific Egyptian production entity still require commercial confirmation.

معيار القبول

Acceptance: finance report can explain customer total, provider charge, margin and pending commission separately.

29. Nuitee pricing/usage costs

الهدف

Keep provider usage costs visible and configurable.

طريقة التنفيذ

Current Nuitee docs state core Rates -> Prebook -> Book is free under ToS/reasonable look-to-book; price-index endpoints are billed per request and Places endpoints per request. Rate limits must be configurable.

ملاحظات/قرارات

Do not accidentally call premium price-index endpoints on every UI scroll.

معيار القبول

Acceptance: search traces show endpoint category and estimated provider cost.

```
HotelSettlementStrategy =  
  OUR_MERCHANT_OF_RECORD
```

Keep strategy behind config/feature flags.

30. Car provider slot

الهدف

Reserve a fully compatible CarProvider slot before selecting the final vendor.

طريقة التنفيذ

Interface must cover search, availability, quote/recheck, booking, cancellation, vehicle details, supplier, pickup/dropoff, drivers, fees, insurance/add-ons, images, and voucher.

ملاحظات/قرارات

Vendor selection criteria: real sandbox, bookable test flow, photos/vehicle details, predictable cancellation, clear currency/POS rules, Egypt support, and transparent commercial terms.

معياري القبول

Acceptance: a dummy adapter/fixture can run the same car workflow before vendor onboarding.

31. Payment orchestrator

الهدف

Create one customer-payment abstraction that routes by paymentRegion and market.

طريقة التنفيذ

PaymentService exposes createPayment, confirmPayment, capture/charge, refund, retrieve, verifyWebhook. PaymentRouter selects gateway by market configuration.

ملاحظات/قرارات

The provider and payment gateway are intentionally independent; Duffel/Nuitee are suppliers, not the customer payment UI.

معياري القبول

Acceptance: switching Egypt gateway implementation does not touch booking services.

32. Egypt payment gateway

الهدف

Start with one Egyptian gateway and keep others as plug-ins.

طريقة التنفيذ

Implement gateway adapter, webhook verification, idempotency, refunds, payment status mapping, settlement reporting and fee extraction.

ملاحظات/قرارات

Before production, confirm merchant account requirements, supported card/wallet methods, EGP settlement, refund timing, chargeback handling and KYC.

معياري القبول

Acceptance: sandbox payment + webhook + refund flow passes end-to-end.

CarProvider contract:

search

quote/revalidate

- book
- cancel
- vehicleDetails
- supplier
- pickup/dropoff
- drivers
- fees
- insurance/addons
- voucher

33. Multi-country payment gateways

الهدف

Allow opening new payment rails by market without modifying travel providers.

طريقة التنفيذ

Payment gateway config: country, currencies, methods, min/max, 3DS, refund support, active, priority, credential reference.

ملاحظات/قرارات

Example configuration can be EG -> Egyptian gateway; SA -> Saudi gateway; AE -> UAE gateway. Do not hard-code country logic in payment service.

معيار القبول

Acceptance: changing market from EG to SA changes only the payment routing/config and localized checkout.

```
PaymentService:  
  createPayment  
  confirmPayment  
  capture  
  refund  
  retrieve  
  handleWebhook
```

PaymentRouter chooses gateway by paymentRegion.

34. Market/POS-aware provider requests

الهدف

Pass market context only where a provider supports/needs it.

طريقة التنفيذ

Create ProviderMarketContextMapper per adapter. Some suppliers use country/POS fields; others do not. Duffel search should not be forced into an invented country parameter.

ملاحظات/قرارات

The market still affects display/payment even when the provider does not vary inventory by market.

معيار القبول

Acceptance: each adapter declares which MarketContext fields it consumes.

```

PaymentGateway {
    createPaymentIntent(...)
    confirm(...)
    refund(...)
    verifyWebhook(...)
    getSettlement(...)
}

```

35. Pricing engine + customer price calculation

الهدف

Centralize base price, taxes/fees, FX and WeTravellers markup. Display one authoritative customer total and a breakdown.

طريقة التنفيذ

Create deterministic PricingEngine input/output. Include pricing rules version and breakdown lines. Create PriceQuote with base, taxes, fees, markup, discounts, payment fee policy, customer currency and expiry.

ملاحظات/قرارات

Do not add margin inside provider adapters; adapters return supplier truth. Do not promise that an FX-converted display price is bookable until provider revalidation.

معياري القبول

Acceptance: same input + same FX snapshot + same pricing version produces the same amount.

Acceptance: checkout displays a clear price snapshot and revalidation outcome.

```

PaymentGatewayRouting {
    EG -> EgyptGateway
    SA -> SaudiGateway
    AE -> UAEGateway
    EU -> EuropeGateway
}

# Example only; activate only after merchant onboarding/approval.

```

36. Payment ledger

الهدف

Record every financial movement.

طريقة التنفيذ

Create immutable ledger entries with bookingId, paymentId, provider, gateway, direction, amount, currency, type, fee, reference and timestamp.

ملاحظات/قرارات

Never update financial history destructively; append adjustments/reversals.

معياري القبول

Acceptance: a booking with payment+refund has a balanced ledger trail.

```

MarketMapper per provider:
Duffel -> consume only supported market fields
Nuitee -> consume supported currency/nationality/market inputs

```

Never invent undocumented provider parameters.

37. Settlement / wallet accounting

الهدف

Track provider balances separately from customer cash.

طريقة التنفيذ

Maintain external provider wallet/top-up references, provider charges, internal payable and settlement status.

ملاحظات/قرارات

Duffel Balance and Nuitee Wallet are external settlement mechanisms; customer money should never be represented as 'provider wallet funds' before settlement actually occurs.

معياري القبول

Acceptance: finance can reconcile customer receipts with provider liabilities and top-ups.

38. Reconciliation

الهدف

Detect mismatches between our DB, payment gateway, provider statements/wallet and bank.

طريقة التنفيذ

Create daily/weekly reconciliation jobs with states MATCHED, MISSING_INTERNAL, MISSING_PROVIDER, AMOUNT_MISMATCH, REFUND_MISMATCH, MANUAL_REVIEW.

ملاحظات/قرارات

Use provider statements/invoices and gateway settlements as evidence sources; never rely solely on local booking rows.

معياري القبول

Acceptance: seeded mismatch fixtures produce actionable reconciliation cases.

```
LedgerEntry {  
  bookingId  
  paymentId  
  provider  
  gateway  
  type  
  direction  
  amount  
  currency  
  externalReference  
  createdAt  
}
```

39. Refund architecture

الهدف

Make refund processing provider- and gateway-independent.

طريقة التنفيذ

RefundService checks booking/provider cancellation result, determines refundable amount, calls gateway refund, updates ledger and emits customer notification.

ملاحظات/قرارات

Store provider refund reference and gateway refund reference separately. Respect provider cancellation windows and gateway settlement rules.

معيار القبول

Acceptance: partial and full refund cases are supported and auditable.

```
SettlementAccount:
  provider
  wallet/balance reference
  topup references
  provider invoice refs
  payable balance
  lastReconciledAt
```

40. Failure recovery

الهدف

Recover safely from payment/bookings split-brain scenarios.

طريقة التنفيذ

Define explicit recovery jobs for PAYMENT_SUCCEEDED + BOOKING_UNKNOWN, BOOKING_CONFIRMED + PAYMENT_UNKNOWN, PRICE_CHANGED, WEBHOOK_MISSED, REFUND_PENDING.

ملاحظات/قرارات

Do not automatically retry money-moving calls without idempotency/query semantics.

معيار القبول

Acceptance: chaos tests prove no duplicate customer charge or duplicate provider booking.

```
ReconciliationJob:
  gateway settlements
  provider statements/invoices
  internal ledger
  -> match
  -> mismatch queue
  -> manual review
```

41. Notifications

الهدف

Turn internal business events into customer communications.

طريقة التنفيذ

Events: PaymentSucceeded, BookingConfirmed, BookingFailed, ScheduleChanged, CancellationConfirmed, RefundInitiated, RefundCompleted.

ملاحظات/قرارات

Notifications consume internal events, never provider webhooks directly.

معيار القبول

Acceptance: duplicate event delivery does not send duplicate customer messages beyond configured dedupe policy.

```
RefundService:
  verify cancellation policy
  calculate refundable amount
  request provider refund/cancel
  request gateway refund
  write ledger reversals
  notify customer
```

42. Admin control center

الهدف

Give operations a safe way to manage providers, markets and bookings.

طريقة التنفيذ

Screens: Providers, Markets, Payment Gateways, Pricing, Bookings, Payments, Refunds, Reconciliation, Webhooks, Errors, Feature Flags.

ملاحظات/قرارات

Do not expose secrets or full payment card data. Use RBAC and audit logs.

معيار القبول

Acceptance: operations can disable a provider/market and view booking/payment timelines without database access.

```
Recovery queue examples:
PAYMENT_SUCCEEDED_BOOKING_UNKNOWN
BOOKING_CONFIRMED_PAYMENT_UNKNOWN
PROVIDER_TIMEOUT
WEBHOOK_MISSED
PRICE_CHANGED
REFUND_PENDING
```

43. Feature flags

الهدف

Roll out providers and markets gradually.

طريقة التنفيذ

Flags: flights_enabled, hotels_enabled, cars_enabled, provider enabled, market enabled, payment gateway enabled, AI booking enabled.

ملاحظات/قرارات

Use server-side flags for booking-critical decisions; client flags may affect UI only.

معيار القبول

Acceptance: disabling a provider stops new searches/bookings but existing bookings remain queryable.

44. Search cache / content cache

الهدف

Reduce cost/latency without serving stale booking prices.

طريقة التنفيذ

Cache static content (airports, airlines, hotel content, images metadata) longer; cache live rates only briefly and tag with expiry.

ملاحظات/قرارات

Never cache successful booking responses as search inventory.

معياري القبول

Acceptance: static content works offline/fast while live pricing always revalidates before booking.

Admin:

Providers | Markets | Payments | Pricing | Bookings |
Refunds | Reconciliation | Webhooks | Errors | Flags

45. Security / API hardening

الهدف

Protect high-value travel and payment flows.

طريقة التنفيذ

Follow OWASP API Security: object-level auth, authentication, property auth, resource limits, function auth, sensitive flow protection, SSRF defenses, config hardening, API inventory and safe consumption of third-party APIs.

ملاحظات/قرارات

Add rate limits, RBAC, input validation, third-party response validation, timeout budgets, allowlisted outbound hosts, secret rotation and dependency scanning.

معياري القبول

Acceptance: security checklist mapped to OWASP API Top 10 passes before production.

Feature flags:

productEnabled
providerEnabled
marketEnabled
paymentGatewayEnabled
bookingEnabled
aiBookingEnabled

46. Privacy / PII handling

الهدف

Protect traveler identity and payment-related data.

طريقة التنفيذ

Classify data, minimize storage, encrypt at rest/in transit, define retention, audit access, support deletion/rectification flows where required, and keep provider-specific passenger fields only as long as necessary.

ملاحظات/قرارات

Egypt's Personal Data Protection Law No. 151 of 2020 applies to personal data processing; get legal review before production for the exact data/controller/processor obligations.

معياري القبول

Acceptance: a data inventory exists and no unnecessary passenger/payment data is stored.

Cache:

STATIC_CONTENT -> long TTL

SEARCH_RESULTS -> short TTL

LIVE_RATES -> very short / provider-defined

BOOKING -> never cache as inventory

47. Testing pyramid

الهدف

Make provider integration safe to change.

طريقة التنفيذ

Unit tests for domain; mapping/contract tests with fixtures; sandbox integration tests; webhook tests; payment tests; end-to-end staging tests; production canary checks.

ملاحظات/قرارات

External tests should be isolated and reproducible; do not make the full CI depend on live vendor availability.

معياري القبول

Acceptance: each provider adapter has deterministic fixture tests and a live sandbox smoke suite.

48. CI/CD and validation

الهدف

Release safely across mobile/backend/admin.

طريقة التنفيذ

Backend: lint, unit, integration, contract, security, build. Flutter: flutter analyze + flutter test. Add migration checks and environment-secret validation.

ملاحظات/قرارات

Never let failing provider contract tests be silently ignored. Use quarantining only with explicit issue tracking.

معياري القبول

Acceptance: CI blocks production deployment on core domain/payment/booking regressions.

Security baseline:

TLS

RBAC

object-level authorization

- rate limits
- input validation
- outbound allow-list
- secret rotation
- dependency scanning
- audit logs

49. Database model

الهدف

Persist enough information to reconstruct booking/finance history.

طريقة التنفيذ

Core tables: users, markets, providers, provider_credentials, provider_capabilities, trips, trip_items, search_sessions, offers, offer_price_snapshots, bookings, booking_events, payments, payment_attempts, refunds, ledger_entries, fx_rates, webhook_events, provider_errors, notifications, audit_logs.

ملاحظات/قرارات

Use provider IDs as external references, not primary domain meaning. Add createdAt/updatedAt and tenant/user ownership where required.

معياري القبول

Acceptance: a booking can be reconstructed from database state plus immutable events.

- PII classification:
- Identity data
- Passenger data
- Payment references
- Documents
- Retention policy per category

50. AI integration boundary

الهدف

Let the universal AI assistant search travel without coupling to vendor APIs.

طريقة التنفيذ

AI tools call SearchOrchestrator/BookingService commands such as searchFlights, searchHotels, searchCars, compareOffers, priceRevalidate, addToTrip. AI never receives provider credentials.

ملاحظات/قرارات

Keep AI output advisory until the user explicitly confirms high-impact actions (payment/booking).

معياري القبول

Acceptance: swapping Duffel for another flight provider does not change AI tool signatures.

- Tests:
- unit
- domain/state-machine
- adapter fixtures
- provider sandbox contract
- webhook

- payment
- reconciliation
- E2E staging

51. Observability + operations/support timeline

الهدف

Make provider and payment failures diagnosable. Give support one coherent view of every booking.

طريقة التنفيذ

Use correlationId/requestId, provider, market, product, endpoint, latency, HTTP status, provider error code, retry count and bookingId. Add dashboards and alerts. Booking timeline should combine search selection, price revalidation, payment, provider request/response IDs, webhook events, booking confirmation, cancellation and refunds.

ملاحظات/قرارات

Redact PII, card data, secrets and sensitive passenger documents from logs. Store raw provider payloads only in protected audit storage with retention controls.

معياري القبول

Acceptance: a support engineer can trace one booking across app, backend, provider, gateway and webhook. Acceptance: support can answer 'who charged whom, at what price, from which provider, and what happened next?'.

CI pipeline:

- flutter analyze
- flutter test
- backend lint/test
- contract tests
- security scan
- build
- migration check

52. Go-live gating

الهدف

Production activation must be controlled, measurable and reversible.

طريقة التنفيذ

Pre-go-live checklist: provider commercial approval, credentials, payment merchant approval, payout/settlement confirmation, privacy/legal review, sandbox E2E, staging E2E, reconciliation, refunds, monitoring, incident runbook, feature flag plan.

ملاحظات/قرارات

Launch one market + one product first; observe before expanding.

معياري القبول

Acceptance: sign-off checklist is completed by engineering, operations/finance and product/legal as applicable.

- Core entities:
- Market
- Provider
- Capability

Trip
TripItem
Offer
PriceSnapshot
Booking
Payment
Refund
LedgerEntry
WebhookEvent
AuditLog

D. Execution plan - phases in strict order

Phase 0 - Baseline and inventory

Implementation steps

1. Inventory the current repository before editing.
2. Record existing Flutter/backend architecture, existing tests, env handling and repositories/services already present.
3. Identify existing booking/search foundations and do not duplicate them.
4. Create a branch or isolated workstream. No delete/replace of unrelated files.

Validation

- git status
- flutter analyze
- flutter test
- backend test suite

Phase 1 - Domain contracts

Implementation steps

5. Create MarketContext, Money, PriceQuote, ProviderCapabilities.
6. Create FlightOffer/HotelOffer/CarOffer normalized envelopes.
7. Create Booking state machine and error taxonomy.
8. Add database migrations for provider and market configuration.

Validation

- unit tests for all value objects and state transitions

Phase 2 - Provider framework

Implementation steps

9. Implement ProviderRegistry, RoutingPolicy, capability checks, provider health, adapter interface and raw external ID metadata.
10. Implement a fake provider adapter and contract fixtures before real vendors.

Validation

- adapter contract tests; routing tests; failover tests

Phase 3 - Pricing + FX

Implementation steps

11. Implement PricingEngine, FxService, price snapshots, expiry, provider cost and markup breakdown.
12. Make customer price deterministic from snapshot + pricing version.

Validation

- pricing unit tests; FX snapshot tests; price-change acceptance

Phase 4 - Duffel test integration

Implementation steps

13. Create Duffel test credentials.

14. Implement search, deterministic test routes, services/seats where used, order/hold, payment tests, 3DS, cancel/change, webhooks.
15. Map provider states into internal state machine.

Validation

- Duffel sandbox E2E; test cards; 200/202 payment scenarios; webhook tests

Phase 5 - Nuitee test integration

Implementation steps

16. Create Nuitee sandbox account/key.
17. Implement hotel content, images, rates, prebook, ACC_CREDIT_CARD sandbox booking, cancellation/webhooks where supported.
18. Add Nuitee cost/rate-limit monitoring.

Validation

- Nuitee sandbox E2E; image edge cases; prebook price change; booking dashboard verification

Phase 6 - Payment abstraction

Implementation steps

19. Implement PaymentGateway interface, PaymentRouter, PaymentIntent/Attempt/Refund entities, gateway webhook handling and ledger integration.
20. Connect one Egyptian gateway in sandbox.
21. Keep the gateway replaceable.

Validation

- gateway sandbox payment + webhook + refund + duplicate callback tests

Phase 7 - Settlement and finance

Implementation steps

22. Implement provider wallet/settlement account tracking, invoices, reconciliation jobs, fee extraction, FX differences and refund reconciliation.
23. Document provider-specific funding flows for Duffel and Nuitee.

Validation

- seeded reconciliation mismatch suite; ledger balance checks

Phase 8 - Market rollout

Implementation steps

24. Activate EG market only with EGP + ar-EG + EG payment routing.
25. Separate display currency from provider currency.
26. Prepare config schema for SA and AE without enabling them.

Validation

- EG market integration tests; cross-currency quote tests

Phase 9 - Cars provider discovery and integration

Implementation steps

27. Select a car provider using the same criteria: sandbox, book, cancel, photos/details, currencies, Egypt/market support, commercial terms.
28. Implement CarProvider only after provider contract is verified.

Validation

- car contract tests + sandbox E2E

Phase 10 - Operations/admin

Implementation steps

29. Add provider, market, payment gateway, feature flag, bookings, refunds, webhooks, errors and reconciliation admin screens.
30. RBAC + audit logs.

Validation

- admin permission tests; safe disable/enable tests

Phase 11 - Security/privacy/observability

Implementation steps

31. Threat model against OWASP API Security Top 10.
32. Implement metrics, traces, redaction, secret scanning, outbound allow-list and privacy inventory.
33. Review Egyptian privacy requirements with legal counsel before production.

Validation

- security checklist, dependency scan, API auth tests, PII redaction tests

Phase 12 - Staging E2E and production gates

Implementation steps

34. Run full booking journeys in staging.
35. Run payment->booking split-brain recovery drills.
36. Verify refunds, statements, webhook recovery, support timeline and reconciliation.
37. Obtain commercial onboarding approval and live credentials from providers/gateway.

Validation

- Go-live checklist signed; production flags remain off until approval

E. Recommended installation / module structure

Backend module layout

```
backend/src/  
  core/  
    errors/  
    money/  
    fx/  
    idempotency/  
    events/  
    observability/  
    security/  
  modules/  
    market/  
      domain/  
      application/  
      infrastructure/  
    travel/  
      domain/  
        flight/  
        hotel/  
        car/  
        booking/  
        offer/  
      application/  
        search/  
        revalidation/  
        booking/  
        cancellation/  
      providers/  
        registry/  
        routing/  
        flights/  
          duffel/  
          wingieone/      # optional future adapter  
        hotels/  
          nuitee/  
        cars/  
          <provider>/  
      infrastructure/  
        cache/  
        persistence/  
        webhooks/  
    pricing/  
    payments/  
      gateway/
```

```
    egypt/  
    saudi/  
    uae/  
  application/  
  ledger/  
  refunds/  
  reconciliation/  
admin/  
notifications/  
ai/
```

Flutter boundary

```
lib/  
  core/  
    money/  
    localization/  
    routing/  
    networking/  
  features/  
    search/  
    flights/  
    hotels/  
    cars/  
    booking/  
    trips/  
    payments/  
    market/  
    ai/  
  repositories/  
    # consume only WeTravellers backend/domain contracts
```

Existing WeTravellers Flutter architecture (feature-first + Riverpod + GoRouter + Repository -> UseCase -> Controller) should be preserved. Provider adapters belong in the backend, not in Flutter.

Environment variables - illustrative

```
APP_ENV=dev  
API_BASE_URL=...  
  
# Provider references (actual secrets resolved from secret storage)  
DUFFEL_MODE=test  
DUFFEL_API_KEY_REF=...  
NUITEE_MODE=sandbox  
NUITEE_API_KEY_REF=...  
  
# Market defaults  
DEFAULT_MARKET=EG  
DEFAULT_CURRENCY=EGP  
DEFAULT_LOCALE=ar-EG
```

```
# Payment gateway references
PAYMENT_GATEWAY_EG_REF=...
PAYMENT_GATEWAY_SA_REF=...

# Feature flags
FLIGHTS_ENABLED=true
HOTELS_ENABLED=true
CARS_ENABLED=false
```

Never paste actual API keys into .env committed to Git. Use local ignored env for development and a secret manager/CI variables for shared environments.

Installation order for the agent

38. Inspect the repository and preserve all existing code. Do not delete directories/files to make the architecture fit.
39. Identify current backend endpoint, database layer, dependency injection, HTTP client, configuration and test framework.
40. Implement domain primitives and migrations first.
41. Add provider interfaces and a fake provider.
42. Add Pricing/FX and state machine.
43. Integrate Duffel in sandbox.
44. Integrate Nuitee in sandbox.
45. Add payment abstraction and Egyptian gateway sandbox.
46. Add ledger/reconciliation.
47. Add market routing for EG.
48. Implement car provider after selection.
49. Add admin and operational controls.
50. Run full test/analyze suite after each phase.
51. Only then start production onboarding/credentials.

F. Provider onboarding and test runbooks

Duffel - test runbook

52. Create Duffel test account and switch dashboard to Developer Test Mode.
53. Create a test token. Test tokens begin with duffel_test_.
54. Implement Offer Requests and search.
55. Use Duffel Airways for reliable sandbox behavior; do not treat its schedules/prices as real market data.
56. Add deterministic route fixtures documented by Duffel: PVD->RAI no flights, JFK->EWR hold offers, LHR->DXB connecting flights, LTN->STN and SEN->STN payment scenarios.
57. Test payment cards including successful, 3DS and insufficient-funds scenarios.
58. Test create order, cancellation/refund and order changes.
59. Persist provider IDs and webhooks into internal booking events.
60. Only after full sandbox pass, activate live account, add billing card if required for platform fees, fund Duffel Balance and issue live token.

Duffel - current fee notes

- Published current Flights pricing: \$3 per confirmed order.
- Managed Content: 1% of confirmed order value.
- Paid ancillary: \$2 per ancillary.
- Search overage rules are provider-policy based; keep them configurable.
- Do not expose Duffel's internal fee as if it were the airline fare. Store it separately in the ledger.

Nuitee - test runbook

61. Create a Nuitee Connect account and obtain a free sandbox key; no credit card is required for the sandbox.
62. Use the sandbox key only against sandbox resources.
63. Implement hotel content first so the UI has names, facilities, ratings and images.
64. Implement Rates search.
65. Implement Prebook to revalidate price/availability.
66. For sandbox, use ACC_CREDIT_CARD as the simplest booking method; sandbox keys are attached to a hidden test account card.
67. Create a booking and verify it in the Nuitee dashboard.
68. Add webhook ingestion and cancellation/refund mapping as supported by the chosen hotel flow.
69. Before production, attach a production account credit card, fund the Nuitee Wallet, configure payout method, and confirm commercial terms.

Nuitee - current fee/usage notes

- Core Rates -> Prebook -> Book is documented as free under the ToS and reasonable look-to-book usage.
- Price index endpoints are currently priced at \$0.05/request.
- Places endpoints are currently priced at \$0.01/request.
- Sandbox limit: 5 requests/second. Current provider docs also list a production limit of 250 requests/second on the rate-limit page; another FAQ page displays 27,000 requests/minute. Treat the rate-limit value as configuration and verify your account-specific quota before load testing.
- Production requires a production key, a valid account credit card and wallet funds for live bookings.

WINGIEONE - alternative flight provider due diligence

- Keep WINGIEONE as a candidate FlightProvider adapter, not a hard dependency.
- Current public product: WINGIEONEAPI provides flight search and booking, webhooks, API keys and sandbox.

- WINGIEONE also exposes a wallet/finance layer with card or wire top-ups, invoices, statements and reconciliation.
- Current public product page marks Hotel and Car as Coming Soon; do not build against those products until official developer access confirms availability.
- Public pricing/fees and detailed sandbox scenarios are not exposed at the same level as Duffel. Treat commercial pricing and Egypt onboarding as an open onboarding item.

G. Market, currency, payment and settlement implementation

Egypt first

```
marketCountry    = EG
displayCurrency  = EGP
locale           = ar-EG
paymentRegion    = EG
posCountry       = EG    # only pass to providers that document POS support
taxRegion        = EG
```

The user-facing market selector changes MarketContext. It must not directly select Duffel/Nuitee/Car provider. Provider routing and payment routing consume the context separately.

Customer checkout flow

```
Search
-> Offer selected
-> Provider revalidation/prebook
-> Create customer PriceQuote (EGP)
-> Customer confirms
-> Payment gateway creates/charges payment in EGP
-> PaymentSucceeded
-> Provider settlement / booking
-> Provider confirmation
-> BookingConfirmed
```

Provider settlement flows

Duffel: use the settlement method enabled for the account. The Balance method is a primary candidate for the WeTravellers Merchant-of-Record model; the provider payment currency remains the offer/order currency, not necessarily EGP.

Nuitee: for WeTravellers-as-Merchant-of-Record, use the documented ACC_CREDIT_CARD strategy after the local customer payment succeeds. Production currently requires a production key, account credit card and wallet funds. The exact payout rail for an Egyptian legal entity must be confirmed during onboarding before production.

Multi-country expansion

Market	Display	Payment region	Gateway	Provider config
EG	EGP	EG	EgyptGateway	Enable Duffel/Nuitee after onboarding
SA	SAR	SA	SaudiGateway	Add only after gateway/provider commercial verification
AE	AED	AE	UaeGateway	Add only after gateway/provider commercial verification
EU*	EUR*	EU*	EuroGateway*	Future configuration; do not assume one gateway fits all countries

The table is configuration, not a promise of provider/payment availability. Each market must pass its own legal, KYC, payout, tax and provider-support checks.

H. Security, privacy and operational gates

Required security controls

- Use OWASP API Security Top 10 2023 as a minimum threat-model checklist.
- Object-level authorization: every booking/payment/trip ID must be authorized for the current user/account.
- Protect sensitive business flows: search, payment, booking and cancellation are rate-limited and abuse-monitored.
- Validate third-party API responses; external APIs are untrusted input.
- Allow-list provider hostnames and use strict timeouts to reduce SSRF risk.
- Do not log card data, CVV, tokens, secret keys or unnecessary passenger documents.
- Verify webhook signatures; reject replayed/old events where the provider supports timestamps/signatures.
- Rotate provider/payment secrets and keep an audit trail of credential changes.

Egypt privacy/legal note

Egypt's Personal Data Protection Law No. 151 of 2020 is part of the Egyptian data-protection framework. The implementation must include a legal/privacy review for WeTravellers' exact role, data categories, transfers, retention and consent/notice requirements before production. The engineering team must not infer legal compliance from the existence of the API integration alone.

I. Master acceptance checklist

- Flutter contains no Duffel/Nuitee/provider secret or direct supplier API call.
- Backend provider interfaces exist for flights, hotels and cars.
- Provider routing is configuration/capability/health aware.
- MarketContext is implemented and EG is the only production-enabled market initially.
- Display currency is separated from provider/payment/settlement currency.
- PricingEngine returns a complete breakdown and stores an FX snapshot.
- Offer expiry and revalidation are enforced.
- Booking state machine prevents unsafe transitions.
- All financial/provider-changing commands are idempotent.
- Webhooks are verified, stored, deduplicated and replay-safe.
- Duffel sandbox end-to-end passes search -> offer -> payment/hold -> order -> cancellation/change scenarios.
- Nuitee sandbox end-to-end passes content -> rates -> prebook -> book and dashboard verification.
- Egyptian payment gateway sandbox passes payment + webhook + refund tests.
- Ledger records customer receipt, gateway fee, provider charge, FX, markup and refunds.
- Reconciliation can detect mismatches.
- A failure drill proves payment success + provider timeout does not create double booking/charge.
- Car provider is selected and integrated only after commercial/sandbox evaluation.
- Admin can disable a provider or market safely.
- Observability can trace a booking end to end without exposing secrets/PII.
- CI passes flutter analyze + flutter test + backend tests + contract/security checks.
- Production credentials and commercial approvals are stored separately from test credentials.
- A written incident/refund/support runbook exists before public booking.

J. Research findings used in this specification

Duffel - verified current facts

- Test mode is a sandbox with separate test/live resources; test tokens begin with duffel_test_. Duffel Airways is a reliable test airline but uses non-realistic schedules/prices.
- Duffel test tooling includes deterministic route scenarios and test cards including 3DS and failed payment cases.
- Duffel payments support balance and card modes; payment resource docs list balance, card and other specialized methods.
- Duffel current published Flights pricing lists \$3 per confirmed order, 1% managed content and \$2 per paid ancillary.
- Duffel go-live documentation says the account must be activated/verified; Balance can then be funded to pay for flights.

Nuitee - verified current facts

- Free sandbox key requires account signup; no credit card is required for the sandbox.
- Sandbox booking can use ACC_CREDIT_CARD against a hidden test card attached to the sandbox key.
- Nuitee supports User Payment, Account Credit Card, Credit Line and custom payment layers.
- Production booking currently requires production API key, account credit card and wallet funds.
- Core Rates -> Prebook -> Book is documented as free under reasonable use; Price Index and Places are billable endpoints.
- Hotel content includes photos, facilities, ratings and room details; mappedRoomId can be used to retrieve room-specific information/photos.
- Nuitee documents weekly commission payouts for confirmed stays; exact payout rail eligibility for an Egyptian legal entity still needs onboarding confirmation.

WINGIEONE - verified current facts

- WINGIEONEAPI is a B2B REST flight search and booking API with webhooks and a sandbox/developer center.
- WINGIEONEAgent includes flight search/booking, wallet, top-ups, invoices and statements.
- The public product page currently marks Hotel and Car as Coming Soon.
- Public pages emphasize B2B flight inventory, wallet/finance and KYC onboarding; detailed public pricing/sandbox cases are less explicit than Duffel.

K. Open questions that must be confirmed before Production

Item	Required confirmation
Nuitee Egypt Production eligibility	Confirm that an Egyptian legal entity can obtain production access without an additional non-Egypt entity requirement.
Nuitee Egypt payout rail	Confirm the exact payout method, supported currencies and bank/settlement rails available for an Egyptian account.
Nuitee commercial/markup terms	Confirm the exact commercial terms, margins, SSP/rate rules and any market/channel restrictions for WeTravellers.
Duffel Egypt payment/settlement	Confirm the operational path for an Egyptian business to fund Duffel Balance and the relevant bank-transfer currency/fees.
Duffel Flight supplier access	Confirm whether any desired airline content needs additional provider-specific approval or commercial onboarding.
Egypt payment gateway	Choose the first gateway only after verifying legal entity/KYC, EGP settlement, cards, Egyptian wallets if required, 3DS, refunds and chargebacks.
Car Provider	Complete a focused due-diligence round using the same sandbox/book/cancel/photo/currency/Egypt checklist.
Legal/privacy	Get legal review of Egypt personal-data obligations, cross-border data transfer, travel-agency/licensing/consumer-protection obligations and terms/refunds.

L. Definition of Done for the first production slice

- 70. EG market enabled with EGP and ar-EG.
- 71. Duffel FlightProvider passes sandbox and staging E2E.
- 72. Nuitee HotelProvider passes sandbox and staging E2E.
- 73. One Egyptian payment gateway passes staging sandbox payment/refund.
- 74. Pricing, FX, ledger and reconciliation are live in staging.
- 75. All provider/webhook/payment secrets are in protected secret storage.
- 76. Admin can disable flights, hotels or payment gateway without deploy.
- 77. Support can trace and recover booking/payment failures.
- 78. Car provider remains disabled until selected and integrated.
- 79. Production commercial/KYC/payout confirmations are documented.
- 80. flutter analyze and flutter test pass; backend suite passes; integration/contract suites pass.
- 81. A limited production canary is released to EG only, with real-time monitoring and rollback flags.

M. Official research sources

Duffel - Test mode: <https://duffel.com/docs/api/overview/test-mode>

Duffel - Test your integration: <https://duffel.com/docs/api/overview/test-your-integration>

Duffel - Payments: <https://duffel.com/docs/api/v2/payments>

Duffel - Payment Intents: <https://duffel.com/docs/api/payment-intents>

Duffel - Pricing: <https://duffel.com/pricing>

Duffel - Getting started: <https://duffel.com/guides/getting-started>

Duffel - Go live: <https://help.duffel.com/hc/en-gb/articles/360019685579-How-do-I-go-live-once-I-ve-built-my-integration>

Nuitee - Getting sandbox key: <https://docs.liteapi.travel/docs/getting-a-sandbox-key>

Nuitee - Implementing payment: <https://docs.liteapi.travel/docs/implementing-payment>

Nuitee - Account Credit Card: <https://docs.liteapi.travel/docs/account-credit-card>

Nuitee - User Payment: <https://docs.liteapi.travel/docs/user-payment>

Nuitee - FAQ: <https://docs.liteapi.travel/docs/faq>

Nuitee - API pricing: <https://docs.liteapi.travel/reference/api-pricing-usage-costs>

Nuitee - Revenue management: <https://docs.liteapi.travel/docs/revenue-management-and-commission>

Nuitee - Hotel details/images: <https://docs.liteapi.travel/docs/displaying-hotel-details>

Nuitee - Room details: <https://docs.liteapi.travel/docs/room-details>

Nuitee - Rate limiting: <https://docs.liteapi.travel/reference/rate-limiting>

Nuitee - Booking workflow: <https://docs.liteapi.travel/reference/workflow>

WINGIEONE - B2B/API/Sandbox: <https://www.wingieone.com/>

OWASP API Security Top 10 2023: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

Egypt NTRA references to Personal Data Protection Law 151/2020: <https://www.tra.gov.eg/>

N. Agent handoff instructions

- Treat this document as the implementation contract for the provider/market/payment architecture.
- Before coding a vendor adapter, read the vendor's current official documentation and confirm any changed endpoint/version/pricing details.
- Do not delete or rename unrelated existing files/directories. If a change is structurally necessary, preserve behavior and add a migration path.
- Implement one phase at a time and show the changed files + tests at the end of each phase.
- Never expose provider keys or payment secrets to Flutter.
- Never skip provider price revalidation before charging the customer.
- Never assume a provider supports a market/currency/payment method unless documented or confirmed by onboarding.
- Never declare Production-ready from sandbox tests alone. Commercial onboarding, payout/settlement, privacy/legal and payment KYC are separate gates.
- At the end of each phase, run the project's existing validation plus the new tests introduced by the phase. For Flutter, run flutter analyze and flutter test.

O. Version 2.0 Finalization Addendum

Purpose: close the remaining execution ambiguities identified during the final review of Version 1.0 and convert the document into the final agent handoff. The 52 core points remain the baseline; this addendum strengthens provider, payment, market, currency, commercial-onboarding, and recovery instructions without changing the previously approved architecture.

O.1 Final decisions - do not reinterpret

- Flights start with Duffel as the first production-oriented adapter. WINGIEONE remains an approved alternative provider candidate and must be implemented behind the same FlightProvider interface if/when commercial terms are better.
- Hotels start with Nuitee Connect as the first hotel adapter. Hotelbeds, Travelgate, Nuitee alternatives, and other suppliers remain replaceable candidates behind HotelProvider.
- Cars remain disabled until a provider passes the same sandbox, content, price, booking, cancellation, settlement, Egypt-market, and commercial evaluation gates.
- The Flutter app never calls a travel provider or payment gateway directly. All credentials, pricing rules, market rules, booking state, reconciliation, and settlement logic remain server-side.
- Market country is not a provider selector. It controls market context; provider routing is a separate policy layer.
- Customer display currency is not the provider settlement currency. Preserve both values and never silently rewrite provider money into the customer currency for provider booking calls.
- The application must support future market-specific gateways without replacing the payment abstraction or booking domain.
- No provider, currency, payout rail, or production feature is considered supported until it is documented by the provider or confirmed in onboarding. Sandbox success is not commercial approval.

O.2 Provider-specific update - Duffel

Current official documentation reviewed on 24 August 2026: Duffel states that businesses can use Duffel regardless of where they are based. Duffel Payments, however, is a separate product currently limited to a published list of supported countries; Egypt is not on that list. When Duffel Payments is unavailable, Duffel explicitly recommends using another PSP and paying Duffel through Duffel Balance via bank transfer. This means the Egypt architecture should NOT depend on Duffel Payments.

- Primary Egypt payment path: customer pays WeTravellers through the selected Egyptian PSP; WeTravellers records the customer payment; WeTravellers funds Duffel Balance; Duffel Balance pays the flight order.
- Keep a separate provider settlement record because customer payment may be EGP while the Duffel order/payment may be in the order/balance currency.
- Duffel payment documentation requires the payment currency to match the order total currency for Balance payments. Never attempt to send EGP to a GBP/USD-priced order unless the provider explicitly returns that currency as the order currency.
- Duffel Payments can be kept as a future optional strategy if a legal entity/market becomes eligible. Do not wire the current Egypt checkout to Duffel Payments.
- When implementing card collection ourselves, use the Egyptian gateway tokenization/hosted components and keep raw PAN/CVV out of WeTravellers servers unless a certified, explicitly approved flow requires otherwise.
- Refunds and order changes must map provider refund currency/amount separately from the customer refund in EGP. The Ledger must record both sides.

O.3 Provider-specific update - Nuitee

Current Nuitee documentation reviewed on 24 August 2026: sandbox signup is free and does not require a credit card; the sandbox can execute the hotel booking flow without real room reservation or real-

money movement. Production requires a production key, a valid account credit card, and wallet funds. Nuitee documents ACC_CREDIT_CARD as the option used when the integrating application is the merchant of record and implements its own customer payment layer.

- Chosen initial hotel payment strategy: WeTravellers is Merchant of Record for the customer payment; customer pays the Egyptian PSP; WeTravellers books Nuitee using ACC_CREDIT_CARD/account funding.
- Keep Nuitee User Payment as a second strategy in the abstraction. It is not the initial Egypt choice because it would move customer payment processing to Nuitee.
- Do not use Nuitee CREDIT (credit line) for the MVP unless a commercial agreement and credit line are explicitly approved. Nuitee documents credit-line setup as requiring a contract and commonly a credit check and/or deposit; it is not a sandbox method.
- Production Nuitee onboarding must confirm, in writing, that the intended Egyptian legal entity can obtain a production key and the exact payout rail available to that entity. Public docs confirm that a payout method must be configured, but they do not publish a country-by-country Egypt payout matrix.
- Nuitee sandbox should be used for content, images, rates, prebook, booking, cancellation, webhook, and ACC_CREDIT_CARD test flows. Preserve transactionId/prebookId values because the Nuitee Payment SDK documentation says they are not recoverable after the prebook response is lost.
- Nuitee production wallet funding, account card, commission, and payout are separate finance concerns and must be represented in the internal ledger/reconciliation model.

O.4 Provider-specific update - WINGIEONE

Current official WINGIEONE page reviewed on 24 August 2026: WINGIEONEAPI provides REST flight search and booking, webhooks, API keys, a sandbox/developer center, and WINGIEONEAgent provides wallet/top-up/invoice/statement operations. The public product page currently lists Flight as ready and Hotel/Car as Coming Soon. Public pricing and detailed sandbox scenarios are not exposed at the same level as Duffel.

- WINGIEONE is a valid secondary FlightProvider candidate, not a dependency. The adapter must use the exact same internal FlightOffer/Booking/Payment/Settlement interfaces as Duffel.
- Before enabling WINGIEONE, obtain written confirmation for: Egyptian-company onboarding, Sandbox access conditions/KYC, booking and cancellation capabilities in sandbox, payment/wallet terms, pricing/fees, supported POS/markets, top-up currencies, payout/settlement rails, and Production approval criteria.
- Do not assume WINGIEONE Hotel or Car APIs exist today; the public page currently marks them Coming Soon.
- If WINGIEONE commercial terms or Egypt/MENA inventory are superior to Duffel after live-like comparison, switch routing policy for the affected market without changing domain or UI code.

O.5 Market / POS / Currency final contract

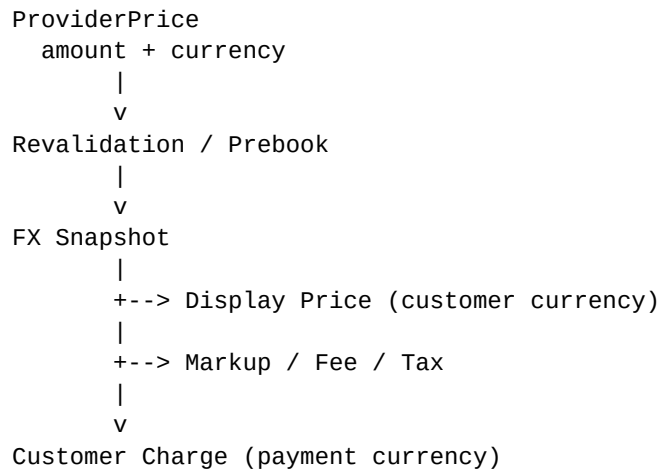
The market selector is a business context object, not a locale-only setting. It must be persisted and propagated to every search/quote/checkout operation that can legally or commercially depend on Point of Sale (POS).

```
MarketContext
  marketCountry    = EG
  displayCurrency  = EGP
  locale           = ar-EG
  paymentRegion    = EG
  taxRegion        = EG
  timezone         = Africa/Cairo
```

Do NOT collapse these into one generic country field.

Required independent concepts: marketCountry, paymentCountry/paymentRegion, residenceCountry, nationality, originCountry, destinationCountry, hotelCountry, displayCurrency, providerCurrency, paymentCurrency, and settlementCurrency. Only capture the fields that are actually needed for the domain, but do not semantically merge them.

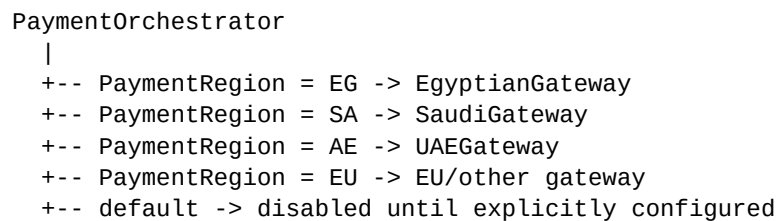
O.6 Pricing contract - final money model



Provider Settlement remains in the provider/order currency.

Every offer price snapshot must retain: providerAmount, providerCurrency, customerAmount, customerCurrency, markupAmount, markupCurrency, feeAmount, taxAmount, fxRate, fxSource, fxTimestamp, pricingVersion, expiresAt, providerId, providerOfferId, and marketContextId. A single flattened price field is prohibited for booking-critical records.

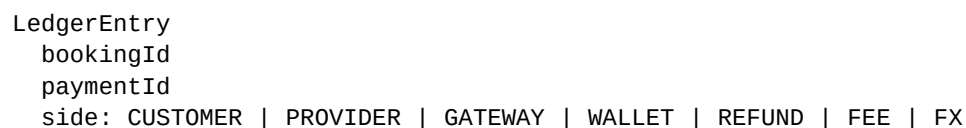
O.7 Payment routing - multi-country design



- Payment gateway choice is based on paymentRegion and business rules, not provider name.
- Gateway adapter interface must expose createPayment, confirmPayment, refundPayment, retrievePayment, verifyWebhook, and idempotency semantics.
- The system must support a hosted/redirect/tokenized payment flow so raw card data is not stored by WeTravellers.
- Gateway availability is market-scoped and feature-flagged. A market can remain search-enabled while checkout is disabled.
- Payment provider identifiers must never leak into customer-facing domain models when a generic PaymentMethod/PaymentStatus is enough.

O.8 Ledger and reconciliation - mandatory

Add a double-entry-style or explicitly traceable transactional ledger before production. The ledger is not optional because travel bookings combine customer payments, provider charges, markup, gateway fees, FX, refunds, and potential wallet balances.



amount
currency
sourceId
occurredAt
metadata

- Every payment attempt gets a stable internal ID and idempotency key.
- Every provider booking maps to exactly one internal booking aggregate, even if retries or webhooks arrive multiple times.
- Every refund stores the original payment reference and the provider refund reference where available.
- Daily reconciliation compares WeTravellers payments, provider statements/wallets, and gateway settlements.
- Finance mismatch states must be visible to Admin and Support; never silently auto-delete or overwrite a mismatch.

O.9 Recovery matrix - mandatory failure cases

Failure scenario	Required behavior
Payment succeeded / booking timeout	Mark PAYMENT_SUCCEEDED + BOOKING_PENDING_RECONCILIATION; query provider before retrying; use idempotency; never blindly book again.
Booking succeeded / webhook missed	Scheduled reconciliation retrieves provider booking state and backfills internal events.
Price changed at prebook	Stop checkout; show new price; require explicit customer confirmation before new payment.
Payment succeeded / booking rejected	Start refund workflow; record provider error and customer-visible refund status.
Cancel requested / provider pending	Keep CANCEL_PENDING; do not tell customer it is refunded until provider/gateway confirms.
Provider unavailable	Keep the search result but disable booking for that offer if price/availability cannot be revalidated; optionally route to an alternate provider only when commercial identity rules permit.
Duplicate webhook	Deduplicate by provider event ID and internal event hash; processing must be idempotent.

O.10 Provider switching / growth playbook

- Never expose Provider A/B/C names in Flutter widgets or domain copy.
- Add a new adapter only after a capability matrix and contract-test suite exist for it.
- Enable new providers by market and service through feature flags and routing weights.
- Start a new provider in shadow or test comparison where allowed: compare inventory count, final price after revalidation, baggage/amenities, cancellation, booking success rate, latency, and supportability.
- Switch only the routing policy when replacing a provider. Do not rewrite domain models or UI.

O.11 Final 52-point coverage audit

All 52 original points remain in sections C and are now reinforced by the V2.0 controls below. The audit is meant for the agent and reviewers to verify that no point was dropped during implementation.

Original points	V2.0 verification
1-10	Provider layer, interfaces, capabilities, routing, aggregation, dedupe, ranking, failover, caching, credentials.
11-20	MarketContext, POS, country semantics, currency separation, FX, pricing, revalidation, state machine, idempotency, webhook foundation.

Original points	V2.0 verification
21-30	Booking model, payment abstraction, gateway routing, settlement, ledger, refund, reconciliation, notifications, support, admin.
31-40	Security, secrets, observability, test environments, provider contract tests, content vs live rate, data provenance, search/booking separation, scale, recovery.
41-52	Provider onboarding, WINGIEONE candidate controls, Duffel/Nuitee production gates, cars provider selection, multi-market enablement, compliance/legal, AI orchestration, final production canary and rollback.

O.12 Do-Not-Assume checklist - mandatory for the coding agent

- Do not assume Egypt is eligible for Nuitee Production payout simply because Sandbox is available. Confirm onboarding and payout rail with Nuitee.
- Do not assume WINGIEONE Sandbox is self-serve or that its public pricing exists. Obtain the exact commercial/onboarding details.
- Do not assume Duffel Payments is available to an Egypt-based entity. The current published supported-country list excludes Egypt; use an external PSP + Duffel Balance unless eligibility changes.
- Do not assume all provider offers can be displayed in EGP or settled in EGP.
- Do not assume nationality, residency, market/POS, payment country, hotel country, and trip origin are interchangeable.
- Do not assume search price equals bookable price. Revalidate/prebook immediately before charging and booking.
- Do not assume a successful payment means a successful booking. Booking must reach a confirmed provider state.
- Do not assume a provider supports cancellation, changes, refunds, seats, baggage, webhooks, or images unless its capability matrix says so.
- Do not persist secrets in Flutter, source control, logs, crash reports, or analytics.
- Do not delete files/directories or replace existing architecture simply to make an adapter easier. Preserve existing behavior and add migration paths.

O.13 Agent execution order - final

Phase	Execution requirement
Phase 0 - Baseline lock	Read current repository architecture, tests, providers, database, existing booking/search foundations. Create a git checkpoint/tag or equivalent. No deletion.
Phase 1 - Domain contracts	Implement/confirm MarketContext, Provider interfaces, Capability model, Offer/Booking/Payment abstractions, pricing snapshot schema.
Phase 2 - Provider infrastructure	Provider registry, routing policy, secret loading, environment separation, webhook gateway, provider health and feature flags.
Phase 3 - Pricing and FX	FX service, markup/fee/tax engine, currency separation, price provenance, expiry, revalidation contracts.
Phase 4 - Duffel Sandbox	Search -> offer -> optional hold -> payment simulation -> order -> cancellation/change -> webhook -> error cases; add contract tests.
Phase 5 - Nuitee Sandbox	Content/images -> rates -> prebook -> ACC_CREDIT_CARD booking -> cancel -> webhook -> dashboard verification; add contract tests.
Phase 6 - Payment abstraction	Gateway interface, Egyptian gateway sandbox,

Phase	Execution requirement
	payment intents/confirm/refund/webhook, idempotency, no raw card storage.
Phase 7 - Finance	Ledger, wallet/provider settlement records, reconciliation, refunds, finance/admin views, mismatch states.
Phase 8 - Recovery and support	Payment/booking recovery jobs, reconciliation jobs, support timeline, manual recovery controls, notifications.
Phase 9 - Cars	Select provider only after evidence-based evaluation; implement CarProvider adapter and full contract tests.
Phase 10 - Market expansion	Add SA/AE/EU market records, gateways, POS rules, currency rules, provider eligibility only after written support confirmation.
Phase 11 - Optional provider competition	Add WINGIEONE or another provider, run comparison and canary, then route by market/provider policy.
Phase 12 - Production gate	Commercial/KYC/payout/settlement approvals, security audit, monitoring, disaster recovery, limited EG canary, rollback flags, full evidence pack.

O.14 Final external verification record

The agent must capture the following evidence before enabling Production:

- Duffel live eligibility and funding instructions for the actual WeTravellers legal entity.
- Nuitee production key eligibility and payout method available to the actual WeTravellers legal entity.
- WINGIEONE onboarding/KYC and exact commercial terms if selected.
- Egypt payment gateway onboarding approval, supported methods/currencies, refunds, settlement timing, and webhook signatures.
- Car provider onboarding and production approval, including Egypt coverage if required.
- Final provider capability matrix and routing policy version.
- Security review and privacy/legal sign-off.
- End-to-end staging evidence for Search -> Revalidate/Prebook -> Payment -> Book -> Webhook -> Confirm -> Cancel/Refund.

O.15 Updated official source record

Duffel Payments: <https://duffel.com/docs/api/v2/payments>

Duffel Collecting and Making Payments: <https://duffel.com/guides/collecting-payments-from-your-customers>

Duffel business country eligibility: <https://help.duffel.com/hc/en-gb/articles/360019685199-Does-it-matter-what-country-my-business-is-based-in>

Duffel Orders: <https://duffel.com/docs/api/orders>

Duffel Order Cancellations: <https://duffel.com/docs/api/order-cancellations>

Nuitee Account Credit Card: <https://docs.liteapi.travel/docs/account-credit-card>

Nuitee Sandbox API Key: <https://docs.liteapi.travel/docs/getting-a-sandbox-key>

Nuitee FAQ: <https://docs.liteapi.travel/docs/faq>

Nuitee Payment Methods: <https://docs.liteapi.travel/docs/implementing-payment>

Nuitee User Payment: <https://docs.liteapi.travel/docs/user-payment>

Nuitee Revenue Management: <https://docs.liteapi.travel/docs/revenue-management-and-commission>

Nuitee Flights Access: <https://docs.liteapi.travel/docs/getting-access-to-flights>

Nuitee Credit Line: <https://docs.liteapi.travel/docs/credit-line>

WINGIEONE: <https://wingieone.com/>

OWASP API Security Top 10: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

O.16 Final handoff instruction

This Version 2.0 document is the final architecture/build contract for the Agent. The Agent must not reinterpret provider-specific commercial facts as assumptions. It must implement the internal contracts first, then provider adapters, then payment/finance, then production onboarding. Any provider-specific fact that is not explicitly confirmed by current provider documentation must be treated as an Open Question and must not be hard-coded.

Validation remains mandatory after every phase: run the project's existing validation plus new tests. For Flutter, run `flutter analyze` and `flutter test`. For backend/provider/payment modules, run unit, integration, contract, webhook, idempotency, recovery, and security tests appropriate to the changed scope.